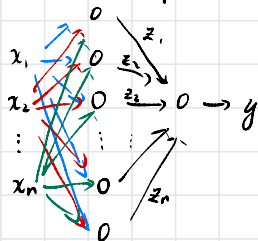


Implement a simple neural network from scratch



(Below we stick to numerator layouts)

i.e. if $z = xy \Rightarrow \frac{\partial z}{\partial x} = y^T, \frac{\partial z}{\partial y} = x^T$
 ↓ scalar

Let us first consider training using SGD (single instance)

write it this way, would be easier to

↑ extend to a batch form

input: $x \in \mathbb{R}^n$

immediate layer: $u = xA_1 + b_1$

$z = \text{relu}(u)$

output: $\hat{y} = zA_2 + b_2$

$L(y, \hat{y}_i) = (y_i - \hat{y}_i)^2$

$\Rightarrow \frac{\partial L}{\partial A_2} = \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial A_2} = 2(\hat{y}_i - y_i) z^T$

$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial b_2} = 2(\hat{y}_i - y_i)$

$\frac{\partial L}{\partial A_1} = \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z} \frac{\partial z}{\partial u} \frac{\partial u}{\partial A_1}$

↓ scalar
 $A_1^T \in \mathbb{R}^{k \times m}$

$\text{diag}(I_{(M_j > 0)}) \in \mathbb{R}^{m \times m}$

$M_j = xA_{1, (..j)}$

$\Rightarrow \frac{\partial L_j}{\partial A_1} = (0^T \ 0^T \dots x^T \ 0^T \dots 0^T)$

$\Rightarrow \frac{\partial L^T}{\partial A_1} = \begin{pmatrix} \frac{\partial L_1}{\partial A_1} \\ \vdots \\ \frac{\partial L_m}{\partial A_1} \end{pmatrix} \in \mathbb{R}^{m \times nm}$

when denominator is a matrix, we flatten it into a long vector:
 column 1
 column 2
 ...
 column m

★ note these transposes are important, we should always

use columns ONLY in denominator to preserve the train rule!

$\frac{\partial L}{\partial b_1^T} = \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z^T} \frac{\partial z^T}{\partial u^T} \frac{\partial u^T}{\partial b_1^T} \rightarrow I_{m \times m}$

how to extend the results into a batch in numpy?

1. make $x \in \mathbb{R}^{1 \times n} \Rightarrow x \in \mathbb{R}^{B \times 1 \times n}$ a 3D tensor

So numpy would return a tensor

2. make $x \in \mathbb{R}^{1 \times n} \Rightarrow x \in \mathbb{R}^{B \times n}$, need to update our chain rule.

input $x \in \mathbb{R}^{B \times n}$

immediate layer: $m \in \mathbb{R}^{B \times m}$ $\nearrow$ $\mathbb{R}^{1 \times m}$

$$m = x A_1 + \mathbf{1}_{B,1} b_1$$

$$z = \text{relu}(m)$$

output: $\hat{y} = z A_2 + b_2 \mathbf{1}_{B,1}$

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^T (y - \hat{y})$$

$$\Rightarrow \frac{\partial \mathcal{L}}{\partial A_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial A_1} = 2(\hat{y} - y)^T z$$

$$\Rightarrow \frac{\partial \mathcal{L}}{\partial b_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b_2} = 2(\hat{y} - y)^T \mathbf{1}_{m,1}$$

$\frac{\partial z_{ij}}{\partial m_{ik}} = \begin{cases} 0 & \text{if } i \neq L \text{ or } L \neq k \\ 1 & \text{if } m_{ik} > 0 \\ 0 & \text{otherwise} \end{cases}$ $\mathbb{R}^{Bm \times Bm}$

$$\Rightarrow \frac{\partial \mathcal{L}}{\partial A_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial m} \frac{\partial m}{\partial A_1}$$

$\frac{\partial \hat{y}}{\partial z} = 2(\hat{y} - y)^T \in \mathbb{R}^{1 \times B}$

$\frac{\partial z}{\partial m} = \begin{cases} 0 & \text{if } L \neq i \\ A_{2,(k,L)} & \text{if } L = i \end{cases} \in \mathbb{R}^{B \times Bm}$

$\frac{\partial m}{\partial A_1} = \begin{cases} 0 & \text{if } k \neq j \\ x_{i,k} & \text{if } k = j \end{cases}$

$\mu_{ij} = \sum_{k=1}^m x_{ik} A_{1,(k,j)}$

$$\Rightarrow \frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial m} \frac{\partial m}{\partial b_1}$$

$\frac{\partial m}{\partial b_1} = \begin{cases} 0 & \text{if } k \neq j \\ 1 & \text{otherwise} \end{cases}$